

| Programación I

Profesor: Carlos Benítez

Material de apoyo pedagógico



Comunitaria de Caacupé, 2023

Bucles o ciclos

Los ciclos o también conocidos como bucles, son una estructura de control esencial al momento de programar. Tanto C como C++ y la mayoría de los lenguajes utilizados actualmente, nos permiten hacer uso de estas estructuras. Un ciclo o bucle permite repetir una o varias instrucciones cuantas veces lo necesitemos, por ejemplo, si quisiéramos escribir los números del uno al mil no tendría sentido escribir mil líneas mostrando un número en cada una, para esto y para muchísimas cosas más, es útil un ciclo, permitiéndonos hacer una misma tarea en una cantidad de líneas muy pequeña y de forma prácticamente automática.



El descubrimiento de los bucles se lo debemos a Ada Byron, así como el de las subrutinas (que no es otra cosa que una función o procedimiento). Está considerada como la primera programadora, y ella misma se autodenominaba analista, lo que no deja de ser sorprendente, ya que el primer ordenador no se construyó hasta un siglo después.

Existen diferentes tipos de ciclos o bucles, cada uno tiene una utilidad para casos específicos.

Los tipos de ciclos en C++:

- ☒ Ciclo while
- ☒ Ciclo do-while
- ☒ Ciclo for

Ciclo **while** en C++.

El ciclo **while** es una estructura cíclica, que nos permite ejecutar una o varias líneas de código de manera repetitiva sin necesidad de tener un valor inicial e incluso a veces sin siquiera conocer cuando se va a dar el valor final que esperamos, los ciclos **while**, no dependen directamente de valores numéricos, sino de valores booleanos, es decir su ejecución depende del valor de verdad de una condición dada, verdadera o falso, nada más. De este modo los ciclos **while**, son mucho más efectivos para condiciones indeterminadas, que no conocemos cuando se van a dar.

¿Cómo funciona un ciclo **while** en C++?

Para comprender mejor el funcionamiento del ciclo **while**, pongamos un buen ejemplo, imaginemos que, por algún motivo, queremos pedirle a un usuario una serie de números cualesquiera y que solo dejará de hacerlo cuando el usuario ingrese un número mayor a 100. Como vemos, aquí no podríamos utilizar un ciclo for, pues no tenemos ni idea de cuándo al usuario se le va a ocurrir ingresar un número mayor que 100, es algo indeterminado para nosotros, sin embargo, el ciclo **while** nos permite ejecutar una acción de forma infinita hasta que se cumpla alguna condición específica, en nuestro caso sería que el número ingresado sea mayor a 100. De modo que, si el usuario ingresa de manera sucesiva los siguientes números 5, 15, 35, 25, 11, 8, 100..., nuestro programa no finalizará, pues ninguno de estos números es mayor que 100, sin embargo, si ingresa el número 101, el programa finalizará inmediatamente.

Sintaxis del ciclo **while** en C++.

Es la sentencia de bucle más sencilla, y sin embargo es tremendamente potente. En C++ se usa la palabra reservada **while** (que significa "mientras"), y la sintaxis es la siguiente:

```
while(condición){  
    sentencia  
}
```

La sentencia es ejecutada repetidamente *mientras* la condición sea verdadera. Si no se especifica condición se asume que es **true**, y el bucle se ejecutará indefinidamente. Si la primera vez que se evalúa la condición resulta falsa, la sentencia no se ejecutará ninguna vez.

Las condiciones no son otra cosa que expresiones de tipo booleano, cualquier otro tipo de expresión se convertirá a tipo booleano, si es posible. Y si no lo es, se producirá un error.

Ejemplo del ciclo **while** en C++.

Quiero pedir que el usuario ingrese números por teclado hasta que ingrese un número negativo.

Queremos hacer que nuestro programa le pida a un usuario una serie de números cualesquiera y que dejara de hacerlo cuando el usuario ingrese un número menor que 0.

```
1  #include<iostream>  
2  using namespace std;  
3  int main(){  
4      int numero;  
5      numero=0;  
6      while(numero >= 0){  
7          cout<<"ingrese un numero: ";  
8          cin>>numero;  
9      }  
10     system("pause");  
11     return 0;  
12 }
```

Ciclo **do-while** en C++.

El ciclo **do-while** es una estructura de control cíclica, que nos permiten ejecutar una o varias líneas de código de forma repetitiva sin necesidad de tener un valor inicial e incluso a veces sin siquiera conocer cuando se va a dar el valor final, hasta aquí son similares a los ciclos **while**, sin embargo el ciclo **do-while** nos permite añadir cierta ventaja adicional y esta consiste que nos da la posibilidad de ejecutar primero el bloque de instrucciones antes de evaluar la condición necesaria, de este modo el ciclo **do-while**, es más efectivo para algunas situaciones específicas. En resumen, un ciclo **do-while**, es una estructura de control cíclica que permite ejecutar de manera repetitiva un bloque de instrucciones

sin evaluar de forma inmediata una condición específica, sino evaluándola justo después de ejecutar por primera vez el bloque de instrucciones.

¿Cómo funciona un ciclo **do-while** en C++?

Para comprender mejor el funcionamiento del ciclo **do-while**, usemos de nuevo el ejemplo anterior del ciclo **while**. Imaginemos entonces que, por algún motivo, queremos pedirle a un usuario una serie de números cualesquiera y que solo dejaremos de hacerlo cuando el usuario ingrese un número mayor a 100. Como vimos anteriormente, esto se puede hacer por medio de un ciclo **while**, pero ahora veremos cómo lo podemos hacer usando un ciclo **do-while** mejorando así un poco nuestro programa, evitando ciertos comandos, tal como se dijo con el ciclo **while**, en efecto aquí estamos en la situación de no tener ni idea de cuándo al usuario se le va a ocurrir ingresar un número mayor que 100, pues es algo indeterminado para nosotros, sin embargo el ciclo **while** y en efecto el **do-while** nos permite ejecutar cierta acción de forma infinita hasta que se cumpla alguna condición específica, en nuestro caso sería que el número ingresado sea mayor a 100. De modo que, si el usuario nos ingresa de manera sucesiva los siguientes números 5, 15, 35, 25, 11, 8, 100..., nuestro programa no finalizara, pues ninguno de estos números es mayor que 100, sin embargo, si nos ingresara el número 101, el programa finalizaría inmediatamente.

Sintaxis del ciclo **do-while** en C++.

Esta sentencia va un paso más allá que el **while**. La sintaxis es la siguiente:

```
do{  
    sentencia  
} while(condición);
```

La sentencia es ejecutada repetidamente mientras la condición resulte verdadera. Si no se especifica condición se asume que es **true**, y el bucle se ejecutará indefinidamente. En otros lenguajes, como **PASCAL**, se usa para el mismo tipo de bucle la estructura "repetir...hasta", es decir la sentencia se repite hasta que se cumpla una determinada condición. La diferencia está en que la lógica es la inversa: la sentencia se repite mientras la condición sea falsa. El resultado es el mismo, en cualquier caso.

A diferencia del bucle **while**, la evaluación de la condición se realiza después de ejecutar la sentencia, de modo que ésta se ejecutará al menos una vez.

Ejemplo del ciclo **do-while** en C++.

Quiero pedir que el usuario ingrese números por teclado hasta que ingrese un número negativo.

Queremos hacer que nuestro programa le pida a un usuario una serie de números cualesquiera y que dejara de hacerlo cuando el usuario ingrese un número menor que 0.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int numero;
5     do{
6         cout<<"ingrese un numero: ";
7         cin>>numero;
8     }while(numero >= 0);
9     system("pause");
10    return 0;
11 }

```

Diferencias entre el ciclo **do-while** en C++ y el ciclo **while**.

La diferencia es que el **do-while**, primero ejecuta la acción y luego evalúa la condición, mientras que el **while** evalúa la condición antes que cualquier cosa. Esto quiere decir, que el ciclo **while** podría no ejecutar ni siquiera una vez lo que tenga en su interior, mientras que el **do-while** te asegura que lo que pongas dentro se ejecute mínimo una vez, pues primero ejecuta y luego evalúa la condición.

Ejemplo de diferencias entre un **do-while** en C++ y un **while**.

Como siempre, un ejemplo puede ayudarnos mucho más a comprender cuándo usar un **do-while** en lugar de un **while** en C++, así que vamos a implementar el siguiente ejemplo:

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     char res;
5     res='S';
6     while(res=='S'){
7         cout<<"Desea ingresar mas datos (S/N): ";
8         cin>>res;
9     }
10    system("pause");
11    return 0;
12 }

```

En este ejemplo la variable `res` se inicializa con el valor de 'S' para que entre en ciclo, porque para entrar en el ciclo **while** la condición tiene que ser verdadera, en caso contrario no haría nada.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     char res;
5     do{
6         cout<<"Desea ingresar mas datos (S/N): ";
7         cin>>res;
8     }while(res=='S');
9     system("pause");
10    return 0;
11 }

```

En este ejemplo la variable `res` solo se define y no es necesario inicializar con el valor de 'S' para que entre en ciclo, porque para entrar en el ciclo **do-while** la condición no se tiene en cuenta la primera vez, a partir de la segunda vez recién el ciclo verifica la condición para continuar si es verdadera, o para finalizar si es falsa.

Ciclo **for** en C++.

El ciclo **for** es el que se conoce como estructura de control de flujo cíclica o simplemente estructura cíclica, este ciclo, como su nombre lo sugiere, nos permite ejecutar una o varias líneas de código de forma iterativa, conociendo un valor específico inicial y otro valor final, además nos permiten determinar el tamaño del paso entre cada "giro" o iteración del ciclo.

En resumen, un ciclo **for** es una estructura de control iterativa, que nos permite ejecutar de manera repetitiva un bloque de instrucciones, conociendo previamente un valor de inicio, un tamaño de paso y un valor final para el ciclo.

¿Cómo funciona un ciclo **for** en C++?

Para comprender mejor el funcionamiento del ciclo **for**, pongamos un ejemplo, supongamos que queremos mostrar los números pares entre el 2 y el 100, si imaginamos un poco como sería esto, podremos darnos cuenta que nuestro ciclo deberá mostrar una serie de números como la siguiente: 2 4 6 8 10 ... 96 98 100. Como podemos verificar, tenemos entonces los componentes necesarios para nuestro ciclo **for**, tenemos un valor inicial que sería el 2, tenemos también un valor final que sería el 100 y tenemos un tamaño de paso que es 2 (los números pares).

Sintaxis del ciclo **for** en C++.

Por último, el bucle para, que usa la palabra reservada **for**. Este tipo de bucle es el más elaborado. La sintaxis es:

```
for( inicialización; condición; incremento ){  
    sentencia  
}
```

La sentencia es ejecutada repetidamente mientras la condición resulte verdadera, o expresado de otro modo, hasta que la evaluación de la condición resulte falsa.

Antes de la primera iteración se ejecutará la iniciación del bucle, que puede ser una expresión o una declaración. En este apartado se suelen iniciar las variables usadas en el bucle. Estas variables también pueden ser declaradas en este punto, pero en ese caso tendrán validez (ámbito) sólo dentro del bucle **for**.

Después de cada iteración se ejecutará el incremento de las variables del bucle. Este incremento también es una sentencia de asignación, y no tiene por qué ser necesariamente un incremento.

En general, podemos considerar que la parte de **inicialización** establece las condiciones iniciales del bucle, la parte de la **condición** establece la condición de salida, y la parte del **incremento**, modifica las condiciones iniciales para establecer las de la siguiente iteración del bucle, o para alcanzar la condición de salida.

Todas las expresiones son opcionales, y si no se especifica la condición se asume que es verdadera.

Ejemplo del ciclo **for** en C++.

Imprimir los números pares comprendidos entre 2 y 100, ambos inclusive.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     cout<<"Los numeros pares entre 2 y 100"<<endl;
5     for( int n = 2; n <= 100; n+=2){
6         cout << n << endl;
7     }
8     system("pause");
9     return 0;
10 }
```

Contenido

Bucles o ciclos2

Los tipos de ciclos en C++:.....2

☒ Ciclo while2

☒ Ciclo do-while2

☒ Ciclo for2

Ciclo **while** en C++.2

 ¿Cómo funciona un ciclo **while** en C++?2

 Sintaxis del ciclo **while** en C++.3

 Ejemplo del ciclo **while** en C++.3

Ciclo **do-while** en C++.3

 ¿Cómo funciona un ciclo **do-while** en C++?4

 Sintaxis del ciclo **do-while** en C++.4

 Ejemplo del ciclo **do-while** en C++.4

 Diferencias entre el ciclo **do-while** en C++ y el ciclo **while**.5

 Ejemplo de diferencias entre un **do-while** en C++ y un **while**.5

Ciclo **for** en C++.6

 ¿Cómo funciona un ciclo **for** en C++?6

 Sintaxis del ciclo **for** en C++.6

 Ejemplo del ciclo **for** en C++.7